

OSSP LAB SKILL PROGRAM1: -

To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Make file.

Code: -

```
#include <stdio.h>

int main ()

{

    printf("=====\n");

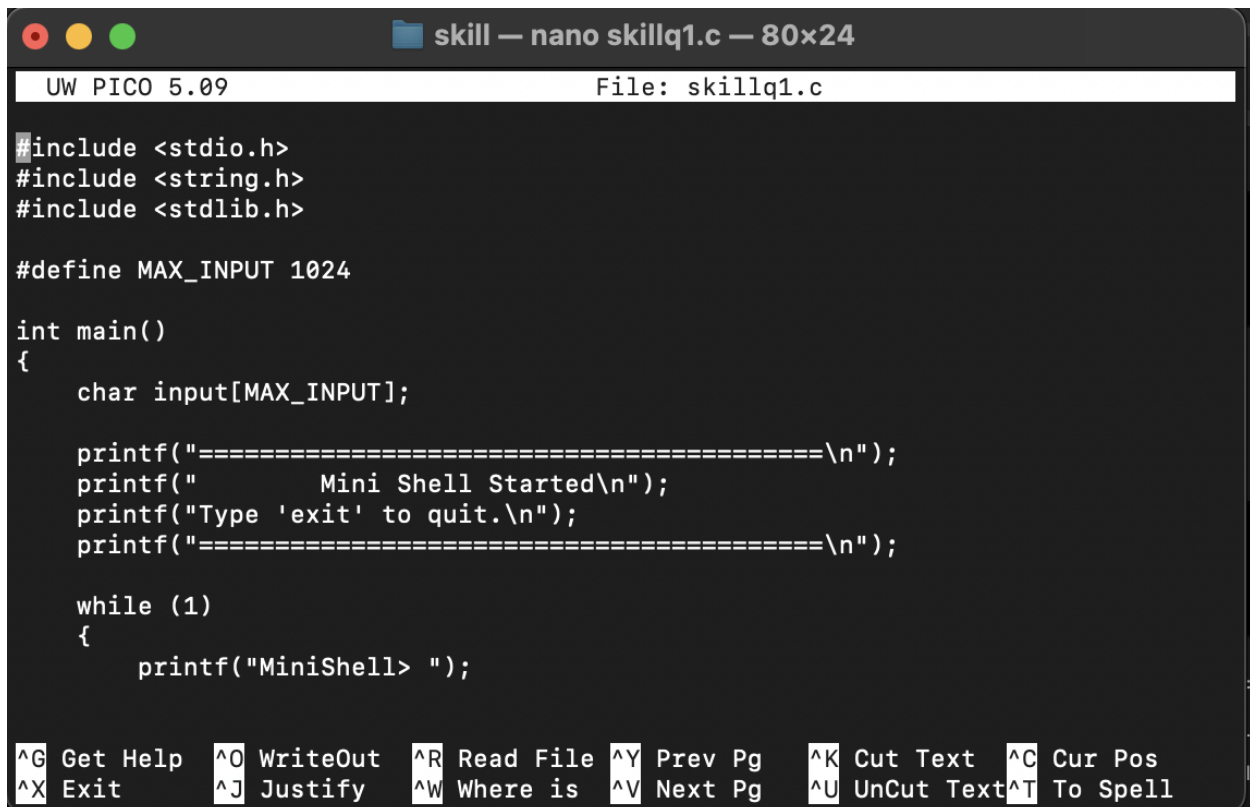
    printf("  Welcome to Mini Shell\n");

    printf("=====\n");

    return 0;

}
```

OUTPUT: -



```
skill — nano skillq1.c — 80x24
UW PICO 5.09 File: skillq1.c

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_INPUT 1024

int main()
{
    char input[MAX_INPUT];

    printf("=====\n");
    printf("      Mini Shell Started\n");
    printf("Type 'exit' to quit.\n");
    printf("=====\n");

    while (1)
    {
        printf("MiniShell> ");
    }
}
```

^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell


```

apple@apples-MacBook-Pro-2 skill % nano skillq1.c
apple@apples-MacBook-Pro-2 skill % gcc skillq1.c -o skillq1
apple@apples-MacBook-Pro-2 skill % ./skillq1
=====
Mini Shell Started
Type 'exit' to quit.
=====
MiniShell> hello
You entered: hello
MiniShell>
MiniShell>
MiniShell> ls -l
You entered: ls -l
MiniShell> pwd
You entered: pwd
MiniShell> exit
Goodbye!

```

OSSP LAB SKILL PROGRAM2: -

To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop.

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

Code: -

```

#include <stdio.h>

#include <string.h>

#include <stdlib.h>

int main()

{

    char input[MAX_INPUT];

    printf("=====\n");

    printf("    Welcome to Mini Shell\n");

    printf("Type 'exit' to quit.\n");

    printf("=====\n");

```

```
while (1)

{

    printf("MiniShell> ");

    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)

    {

        printf("\nExiting shell...\n");

        break;

    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)

    {

        continue;

    }

    if (strcmp(input, "exit") == 0)

    {

        printf("Goodbye!\n");

        break;

    }

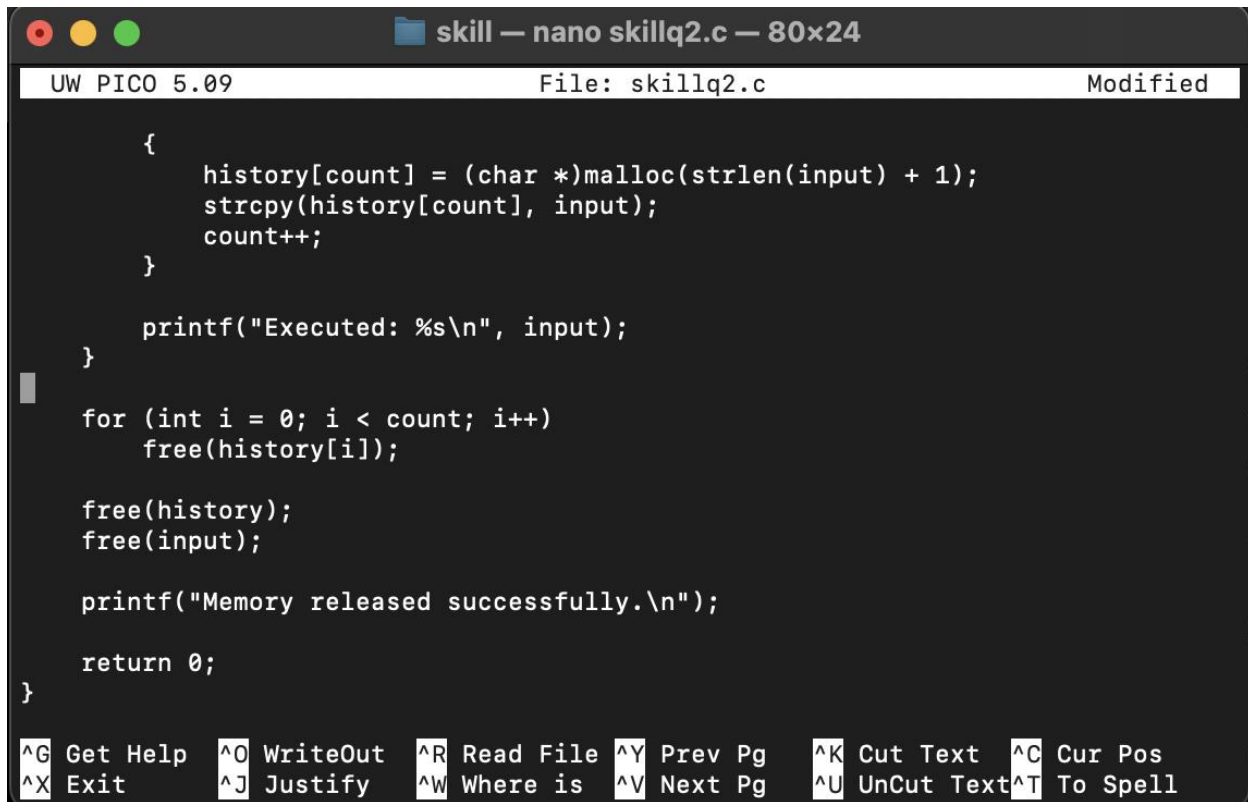
    printf("Command entered: %s\n", input);

}
```

```
Return 0;

}
```

OUTPUT: -



```
skill — nano skillq2.c — 80x24
UW PICO 5.09 File: skillq2.c Modified

    {
        history[count] = (char *)malloc(strlen(input) + 1);
        strcpy(history[count], input);
        count++;
    }

    printf("Executed: %s\n", input);
}

for (int i = 0; i < count; i++)
    free(history[i]);

free(history);
free(input);

printf("Memory released successfully.\n");

return 0;
}

^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Pg   ^K Cut Text   ^C Cur Pos
^X Exit      ^J Justify   ^W Where is   ^V Next Pg   ^U UnCut Text ^T To Spell
```

```
apple@apples-MacBook-Pro-2 skill % ./skillq2
===== Mini Shell =====
Type 'history' to view history.
Type '!!' to execute last command.
Type '!n' to execute command number n.
Type 'exit' to quit.

MiniShell> histtory
Executed: histtory
MiniShell> history

Command History:
1 : histtory

MiniShell> pwd
Executed: pwd
MiniShell> !!
Previous Command: pwd
MiniShell> 
```

OSSP SKILL PROGRAM3: -

To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

Code: -

```
#include <stdio.h> #include <stdlib.h> #include <string.h>
```

```
#define MAX_INPUT 1024 #define INITIAL_HISTORY 5
```

```
int main() { char *input; char **history;
```

```
int historyCount = 0;
```

```
int historySize = INITIAL_HISTORY;
```

```
input = (char *)malloc(MAX_INPUT * sizeof(char));
```

```
if (input == NULL)
```

```
{
```

```
    printf("Memory allocation failed!\n");
```

```
    return 1;
```

```
}
```

```
history = (char **)malloc(historySize * sizeof(char *));
```

```
if (history == NULL)
```

```
{
```

```
    printf("Memory allocation failed!\n");
```

```
    free(input);
```

```
    return 1;
```

```
}
```

```
printf("=====\n");
```

```
printf("    Mini Shell - Q3\n");
```

```
printf("=====\n");
```

```
printf("Commands:\n");
printf(" history - Show command history\n");
printf(" !!    - Show previous command\n");
printf(" !n    - Recall command number n\n");
printf(" exit   - Exit shell\n\n");
```

```
while (1)
{
    printf("MiniShell> ");
    fflush(stdout);

    if (fgets(input, MAX_INPUT, stdin) == NULL)
    {
        printf("\n");
        break;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        continue;
    }

    if (strcmp(input, "exit") == 0)
    {
        break;
    }

    if (strcmp(input, "history") == 0)
    {
        if (historyCount == 0)
        {
            printf("No commands in history.\n");
        }
        else
        {
            printf("\nCommand History:\n");
```

```
    for (int i = 0; i < historyCount; i++)
    {
        printf("%d %s\n", i + 1, history[i]);
    }

    printf("\n");
}

continue;
}

if (strcmp(input, "!!") == 0)
{
    if (historyCount == 0)
    {
        printf("No previous command.\n");
    }
    else
    {
        printf("Previous command: %s\n",
            history[historyCount - 1]);
    }

    continue;
}

if (input[0] == '!' && input[1] != '\0')
{
    int number = atoi(&input[1]);

    if (number >= 1 && number <= historyCount)
    {
        printf("Command %d: %s\n",
            number,
            history[number - 1]);
    }
    else
```



```

    {
        printf("Invalid history number.\n");
    }

    continue;
}

if (historyCount >= historySize)
{
    historySize *= 2;

    char **temp = realloc(
        history,
        historySize * sizeof(char *)
    );

    if (temp == NULL)
    {
        printf("Unable to resize history.\n");
        break;
    }

    history = temp;
}

history[historyCount] =
    (char *)malloc(strlen(input) + 1);

if (history[historyCount] == NULL)
{
    printf("Memory allocation failed.\n");
    break;
}

strcpy(history[historyCount], input);

historyCount++;

```

```
    printf("Command stored: %s\n", input);
}

for (int i = 0; i < historyCount; i++)
{
    free(history[i]);
}

free(history);
free(input);

printf("Memory released successfully.\n");

return 0;

}
```

OUTPUT: -

```
skill — nano skillq3.c — 80x24
UW PICO 5.09 File: skillq3.c Modified

    {
        printf("Arguments:\n");

        for (int i = 1; i < tokenCount; i++)
        {
            printf("  Arg %d : %s\n", i, tokens[i]);
        }
    }
    else
    {
        printf("No Arguments\n");
    }
}

printf("-----\n");
}

printf("Shell Closed.\n");

^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell
```

```
apple@apples-MacBook-Pro-2 skill % nano skillq3.c
apple@apples-MacBook-Pro-2 skill % gcc skillq3.c -o skillq3
apple@apples-MacBook-Pro-2 skill % ./skillq3
===== Mini Shell Parser =====
Type 'exit' to quit.

MiniShell> ls

Parsed Tokens:
Token 1 : ls

Command Structure:
Command : ls
No Arguments
-----

MiniShell> exit
Shell Closed.
```

OSSP SKILL PROGRAM4: -

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

Code: -

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <ctype.h>

#define MAX_INPUT 1024

#define MAX_TOKENS 100

int main()

{

    char input[MAX_INPUT];

    char *tokens[MAX_TOKENS];

    int tokenCount;

    printf("=====\n");

    printf("    Mini Shell Parser - Q4\n");

    printf("=====\n");

    printf("Type 'exit' to quit.\n\n");

    while (1)

    {

        printf("MiniShell> ");

        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
```

```
        break;

input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0)
{
    printf("Empty command.\n");

    continue;
}

if (strcmp(input, "exit") == 0)

    break;

tokenCount = 0;

char *token = strtok(input, " \t");

while (token != NULL && tokenCount < MAX_TOKENS - 1)
{

    tokens[tokenCount] = token;

    tokenCount++;

    token = strtok(NULL, " \t");

}

tokens[tokenCount] = NULL;

printf("\nParsed Tokens:\n");

for (int i = 0; i < tokenCount; i++)
{

    printf("Token %d : %s\n", i + 1, tokens[i]);
```

```
}

printf("\nCommand Structure:\n");

if (tokenCount > 0)

{

    printf("Command : %s\n", tokens[0]);

    if (tokenCount > 1)

    {

        printf("Arguments:\n");

        for (int i = 1; i < tokenCount; i++)

        {

            printf("Arg %d : %s\n", i, tokens[i]);

        }

    }

    else

    {

        printf("No arguments.\n");

    }

}

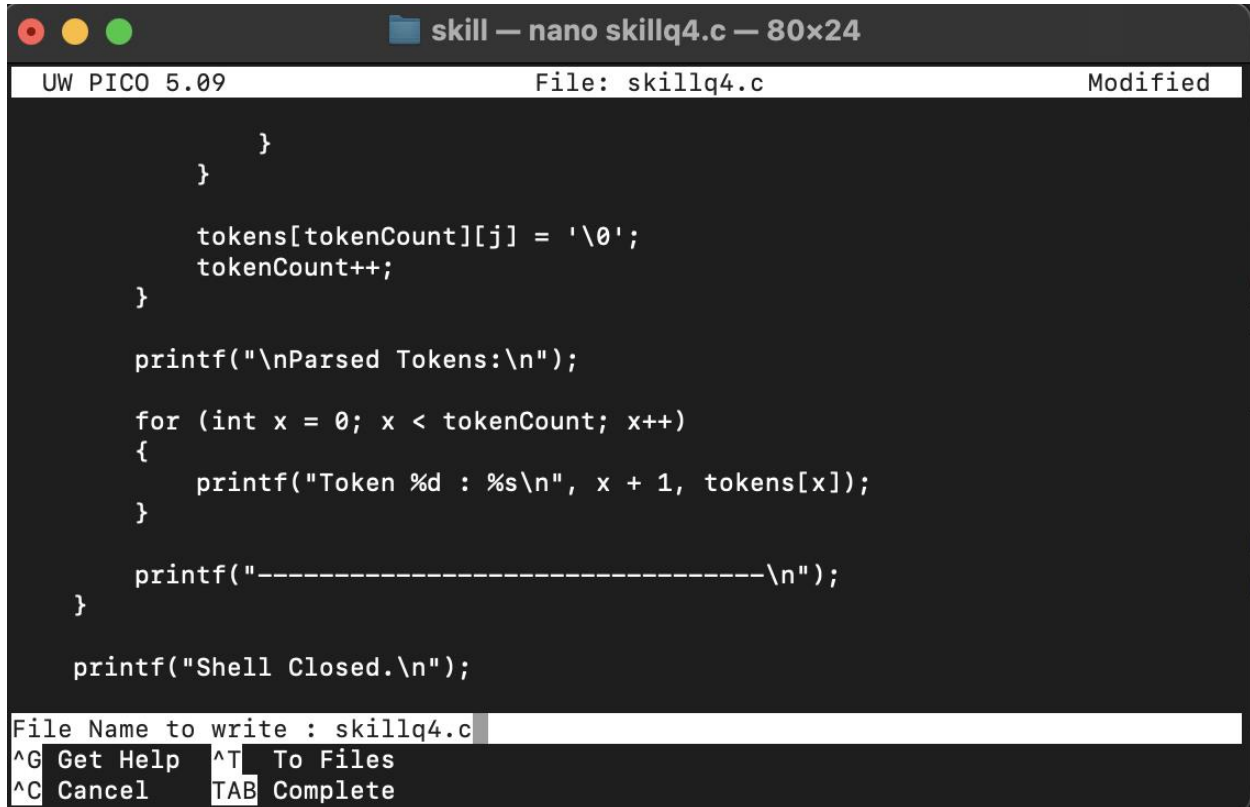
printf("-----\n");

}

printf("Shell Closed.\n");
```

```
    return 0;
}
```

OUTPUT: -



The screenshot shows a terminal window titled "skill — nano skillq4.c — 80x24". The window contains C code for parsing tokens. The code includes a loop to process tokens, a printf statement to display the parsed tokens, and another loop to print each token individually. The code is as follows:

```
    }
}

tokens[tokenCount][j] = '\0';
tokenCount++;
}

printf("\nParsed Tokens:\n");

for (int x = 0; x < tokenCount; x++)
{
    printf("Token %d : %s\n", x + 1, tokens[x]);
}

printf("-----\n");
}

printf("Shell Closed.\n");
```

At the bottom of the window, there is a status bar with the text "File Name to write : skillq4.c". Below this, there are three lines of text: "^G Get Help", "^T To Files", and "^C Cancel", followed by "TAB Complete".

```

apple@apples-MacBook-Pro-2 skill % nano skillq4.c
apple@apples-MacBook-Pro-2 skill % gcc skillq4.c -o skillq4
apple@apples-MacBook-Pro-2 skill % ./skillq4
===== Mini Shell Quote Parser =====
Type 'exit' to quit.

MiniShell> ls

Parsed Tokens:
Token 1 : ls
-----

MiniShell> Hello world

Parsed Tokens:
Token 1 : Hello
Token 2 : world
-----

MiniShell> █

```

OSSP SKILL PROGRAM5: -

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

Code: -

```

#include <stdio.h>

#include <string.h>

#include <stdlib.h>

#include <ctype.h>

```

```

#define MAX_INPUT 1024

```

```

#define MAX_TOKENS 100

```

```

int main()

```

```

{

```



```
char input[MAX_INPUT];
char tokens[MAX_TOKENS][MAX_INPUT];
int tokenCount;

printf("=====\n");
printf("    Mini Shell Parser - Q5\n");
printf("=====\n");
printf("Type 'exit' to quit.\n\n");

while (1)
{
    printf("MiniShell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "exit") == 0)
        break;

    if (strlen(input) == 0)
        continue;

    tokenCount = 0;
```

```
int i = 0;

while (input[i] != '\0' && tokenCount < MAX_TOKENS)
{
    while (isspace(input[i]))
        i++;

    if (input[i] == '\0')
        break;

    int j = 0;

    if (input[i] == '\n')
    {
        i++;

        while (input[i] != '\n' && input[i] != '\0')
        {
            tokens[tokenCount][j++] = input[i++];
        }

        if (input[i] == '\n')
            i++;
    }
    else if (input[i] == '"')
    {

```

```

i++;

while (input[i] != '"' && input[i] != '\0')
{
    if (input[i] == '$')
    {
        i++;

        char variable[100];

        int k = 0;

        while (isalnum(input[i]) || input[i] == '_')
        {
            variable[k++] = input[i++];
        }

        variable[k] = '\0';

        char *value = getenv(variable);

        if (value != NULL)
        {
            strcpy(&tokens[tokenCount][j], value);
            j += strlen(value);
        }
    }
}

```

```

        else
        {
            tokens[tokenCount][j++] = input[i++];
        }
    }
    if (input[i] == "")
        i++;
}
else
{
    while (input[i] != '\0' && !isspace(input[i]))
    {
        tokens[tokenCount][j++] = input[i++];
    }
}
tokens[tokenCount][j] = '\0';
tokenCount++;
}

printf("\nParsed Tokens:\n");
for (int i = 0; i < tokenCount; i++)
{
    printf("Token %d : %s\n", i + 1, tokens[i]);
}

printf("-----\n");
}

printf("Shell Closed.\n");

```

```
return 0;
```

OUTPUT: -

```
apple@apples-MacBook-Pro-2 skill % nano skillq4.c
apple@apples-MacBook-Pro-2 skill % gcc skillq4.c -o skillq4
apple@apples-MacBook-Pro-2 skill % ./skillq4
===== Mini Shell Quote Parser =====
Type 'exit' to quit.
```

```
MiniShell> ls
```

```
Parsed Tokens:
```

```
Token 1 : ls
```

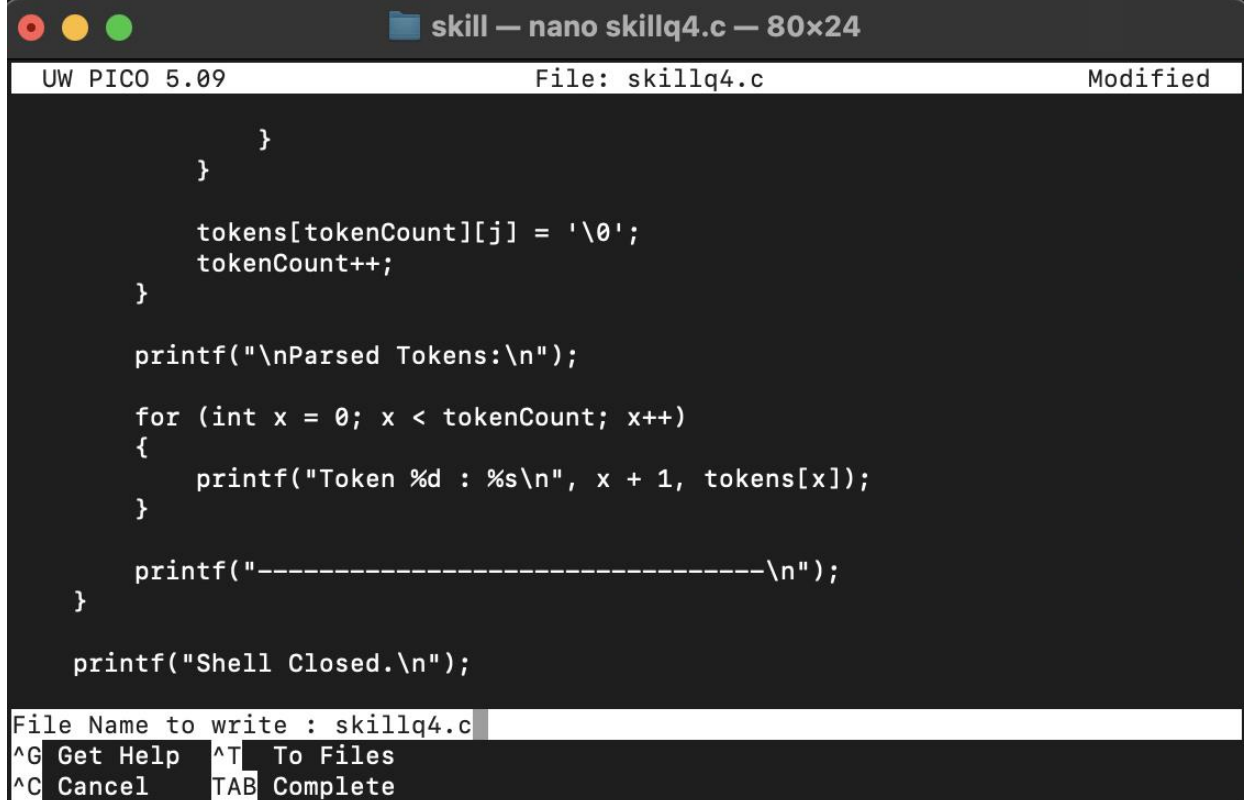
```
MiniShell> Hello world
```

```
Parsed Tokens:
```

```
Token 1 : Hello
```

```
Token 2 : world
```

```
MiniShell> █
```



```
skill — nano skillq4.c — 80x24
UW PICO 5.09 File: skillq4.c Modified

    }
}

tokens[tokenCount][j] = '\0';
tokenCount++;
}

printf("\nParsed Tokens:\n");

for (int x = 0; x < tokenCount; x++)
{
    printf("Token %d : %s\n", x + 1, tokens[x]);
}

printf("-----\n");
}

printf("Shell Closed.\n");

File Name to write : skillq4.c
^G Get Help ^T To Files
^C Cancel TAB Complete
```